

RFID-Based Electronic Voting Machine with Interactive Candidate Selection and Multimodal Feedback

A Human-Computer Interaction Project Using Arduino Uno, RC522 RFID, and SSD1306 OLED Display

Arafat Hossain (23-50277-1) | Rajarshi Mondal (23-50279-1) | Al Imran Alif (23-50273-1) | Tousif Tarik (23-53577-3)
American International University-Bangladesh (AIUB)

Supervisor: DR. MUHAMMAD FIROZ MRIDHA



Introduction

Electronic voting systems represent a critical intersection of Human-Computer Interaction (HCI) and embedded systems design. While large-scale electronic voting infrastructure is complex and costly. This project presents the design and implementation of an RFID-based electronic voting machine built on an Arduino Uno R3 microcontroller. The system uses RFID cards as voter identification tokens, a push button for candidate selection and vote confirmation, an OLED display for real-time visual feedback, and a buzzer with LED for audio-visual response. The prototype demonstrates that meaningful HCI design principles can be implemented effectively even within the strict computational and memory constraints of embedded hardware.

KEY MOTIVATIONS:

- Electronic voting removes illegibility issues of paper ballots.
- Embedded systems allow cost-effective, portable voting terminals.
- HCI principles (feedback, constraints, error prevention) are directly testable in physical interactive systems.
- RFID provides a familiar, fast, and contactless identification method.

Methods

HARDWARE COMPONENTS

The system was built using the components like:

- Arduino Uno R3 (ATmega328P, 32KB flash)
- RC522 RFID Module (SPI, 13.56 MHz, MIFARE)
- SSD1306 OLED Display (128×64 pixels, I2C)
- Active Buzzer (5V)
- Standard 5mm LED with 220Ω resistor
- Tactile push button
- Breadboard, jumper wires

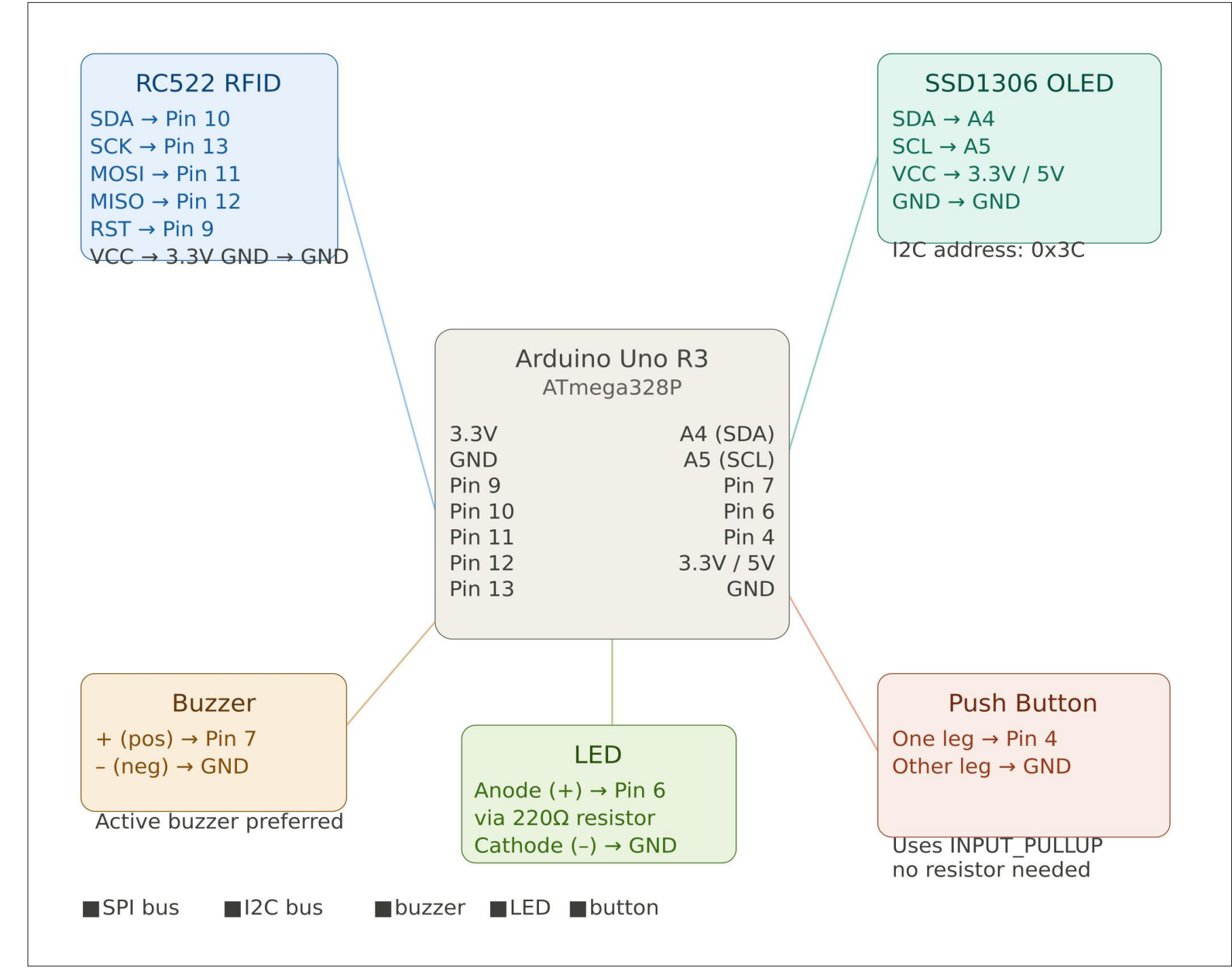


Figure 1. Wiring diagram showing connections between Arduino Uno, RC522 RFID module (SPI bus, pins 9-13), SSD1306 OLED (I2C bus,A4/A5), buzzer (pin 7), LED (pin 6), and push button (pin 4).

SYSTEM ARCHITECTURE - STATE MACHINE

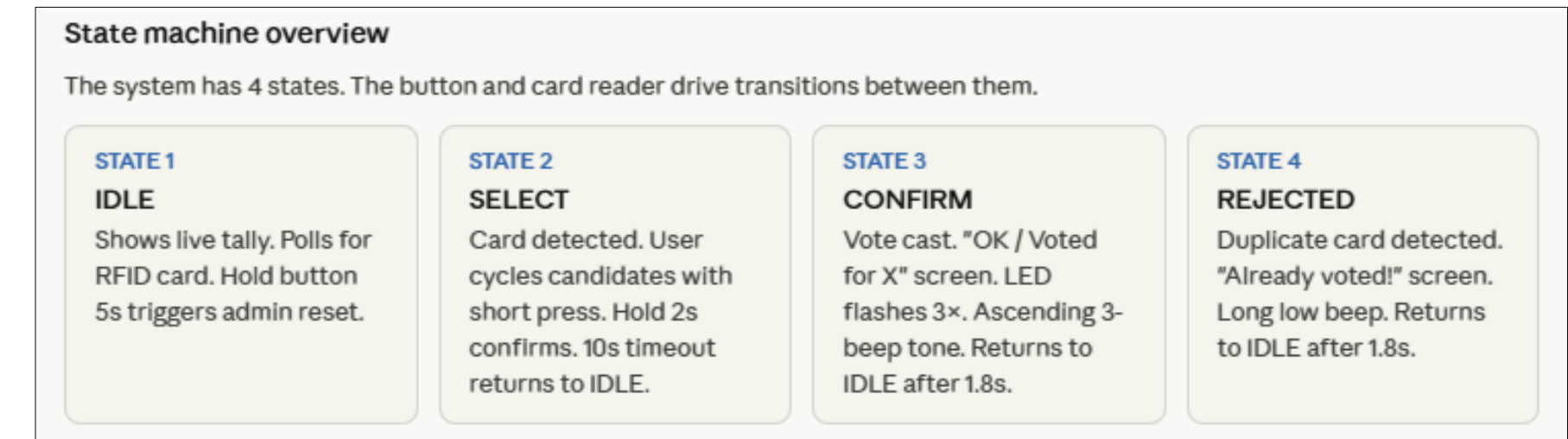


Figure 2. Finite state machine showing the four system states and the events that trigger transitions between them.

DOUBLE-VOTE PREVENTION

Each RFID card has a unique 4-byte UID. On successful vote confirmation, the card's UID is stored in a byte array in SRAM (up to 30 entries). On every subsequent scan, the UID is compared against all stored entries using memcmp(). A match immediately triggers STATE_REJECTED before the voter can reach the selection screen, making double voting impossible without physically presenting a different card.

Data Analysis

HCI PRINCIPLES EVALUATED

The design was analyzed against Nielsen's 10 Usability Heuristics and standard HCI feedback loop principles:

Heuristic	Implementation in This System
Visibility of system status	Tally always shown in IDLE state. Progress bar displayed during hold-to-confirm action.
Match system to real world	“Select → Confirm” interaction mirrors the physical ballot and ballot-box mental model.
User control and freedom	10-second timeout cancels selection. Releasing the button before 2 seconds cancels confirmation.
Error prevention	2-second hold prevents accidental confirmation.UID registry prevents double voting.
Recognition over recall	Candidate list is displayed explicitly.No memorization required from users.
Aesthetic and minimalist design	OLED layout shows only necessary informations. Bar chart reduces cognitive load compared to raw numbers.

BUTTON INTERACTION ANALYSIS

A single push button serves three distinct functions, disambiguated by duration and system state:

Function	Duration	State Required	Guarded By
Cycle candidate	< 500 ms	STATE_SELECT	btnLongFired flag
Confirm vote	≥ 2000 ms	STATE_SELECT	btnLongFired flag
Admin reset	≥ 5000 ms	STATE_IDLE	btnLongFired flag

The btnLongFired boolean flag prevents the long-press action from firing repeatedly while held, and prevents the short-press from triggering falsely when the user releases after a confirmed hold. This is a non-blocking implementation using millis() — no delay() calls are used, keeping the main loop responsive at all times..

Results

IMPLEMENTED FEATURES (VERIFIED WORKING)

- ✓ RFID card scan triggers candidate selection mode
- ✓ Push button cycles through candidates with wrap-around
- ✓ 2-second hold confirms vote with visual progress bar feedback
- ✓ Double-vote prevention via UID registry (tested with same card)
- ✓ Live bar chart tally updates after every vote
- ✓ Distinct buzzer tones for success, rejection, cycling, timeout, reset
- ✓ LED flashes on successful vote confirmation
- ✓ 10-second timeout in selection mode (no vote if abandoned)
- ✓ Admin reset via 5-second hold in IDLE state
- ✓ All 5 OLED screens rendering correctly



Figure 3. OLED display showing (left) the live tally bar chart in IDLE state, and (right) the candidate selection screen with highlighted row during active voting.

4.2 OBSERVED INTERACTION BEHAVIOR

- Typical voting session (card scan → select → confirm): ~5–8 seconds
- RC522 card read latency: < 100ms
- Button cycle response: immediate (next loop iteration, ~1ms)
- Duplicate card detection: immediate on scan, before selection screen
- Admin reset: triggered reliably at 5-second threshold

Conclusion

This project demonstrates that important HCI principles such as multimodal feedback, error prevention, system visibility, and user control can be effectively implemented even on the limited hardware of an Arduino Uno R3. Unlike basic RFID systems, it separates user identification from vote confirmation, ensuring deliberate voter intent. The hold-to-confirm mechanism with a visual progress bar helps prevent accidental voting. Multimodal feedback using the OLED display, buzzer, and LEDs improves usability and accessibility by communicating system states through multiple channels. However, the current system stores data only in volatile SRAM, so votes are lost during power failure. Future improvements could include EEPROM-based storage, RTC-based vote logging, wireless result transmission, and usability testing with real users for quantitative HCI evaluation.

References

- Nielsen, J. (1994). 10 Usability Heuristics for User Interface Design. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>
- Norman, D. A. (2013). The Design of Everyday Things (Revised and Expanded Edition). Basic Books.
- Dix, A., Finlay, J., Abowd, G. D., & Beale, R. (2004). Human-Computer Interaction (3rd Edition). Pearson Education.
- Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., & Elmqvist, N. (2016). Designing the User Interface: Strategies for Effective Human-Computer Interaction (6th Edition). Pearson.
- MFRC522 Arduino Library. GithubCommunity. <https://github.com/miguelbalboa/rfid>
- Adafruit SSD1306 Library. Adafruit Industries. https://github.com/adafruit/Adafruit_SSD1306
- Arduino Reference — millis(), digitalRead(), INPUT_PULLUP.
- <https://www.arduino.cc/reference/en/>

Acknowledgement

The authors would like to express sincere gratitude to Dr. Muhammad Firoz Mridha for guidance and feedback throughout the development of this project. We also thank AIUB for giving us this platform to showcase our project. Special thanks to the open-source Arduino community, particularly the contributors to the MFRC522 and Adafruit SSD1306 libraries, whose work made this project possible.